

Phase 5: Advanced Security and Monitoring Infrastructure (Week 5):

1. Implement Access Control using SELinux or AppArmor, with documentation showing how to track and report on access control settings.?

Ans: Ubuntu Server sticks with AppArmor right out of the box. It's the go-to choice for mandatory access control here, and you don't have to mess with the kernel to get it working.

Command use:

Sudo systemctl status apparmor,

```
all packages are up to date.
ramesh@rameshserver:~$ sudo systemctl status apparmor
• apparmor.service - Load AppArmor profiles
  Loaded: loaded (/usr/lib/systemd/system/apparmor.service; enabled; preset: enabled)
  Active: active (exited) since Tue 2025-12-23 23:23:16 UTC; 5min ago
  Docs: man:apparmor(7)
        https://gitlab.com/apparmor/apparmor/wikis/home/
  Process: 539 ExecStart=/lib/apparmor/apparmor.systemd reload (code=exited, status=0/SUCCESS)
  Main PID: 539 (code=exited, status=0/SUCCESS)
  CPU: 514ms

Dec 23 23:23:15 rameshserver systemd[1]: Starting apparmor.service - Load AppArmor profiles...
Dec 23 23:23:15 rameshserver apparmor.systemd[539]: Restarting AppArmor
Dec 23 23:23:15 rameshserver apparmor.systemd[539]: /lib/apparmor/apparmor.systemd: 148: [: Illegal number: yes
Dec 23 23:23:15 rameshserver apparmor.systemd[539]: Reloading AppArmor profiles
Dec 23 23:23:16 rameshserver systemd[1]: Finished apparmor.service - Load AppArmor profiles.
```

Sudo aa-status:

```
qutebrowser
rootlesskit
rpm
rssguard
runc
sbuid
sbuid-abort
sbuid-adduser
sbuid-apt
sbuid-checkpackages
sbuid-clean
sbuid-creatchroot
sbuid-destroychroot
sbuid-distupgrade
sbuid-hold
sbuid-shell
sbuid-unhold
sbuid-update
sbuid-upgrade
scide
signal-desktop
slack
slirp4netns
steam
stress-ng
surfshark
systemd-coredump
thunderbird
toybox
trinity
tup
tuxedo-control-center
userbindmount
uwsgi-core
vdens
virtiofsd
vivaldi-bin
vpns
vscode
wike
wpcorn
1 processes have profiles defined.
1 processes are in enforce mode.
  /usr/sbin/rsyslogd (784) rsyslogd
0 processes are in complain mode.
0 processes are in prompt mode.
0 processes are in kill mode.
0 processes are unconfined but have a profile defined.
0 processes are in mixed mode.
```

With AppArmor, you set the rules for what each application can and can't do by loading specific security profiles. To check if everything's running as expected, you can use `aa status`. That shows you which profiles are active and confirms that AppArmor is doing its job, keeping those access rules in place.

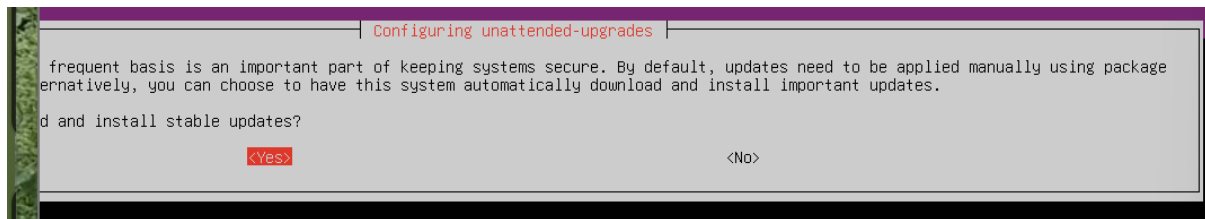
2. Configure automatic security updates with evidence of implementation?

Configuration Steps

Install unattended upgrades:

```
ramesh@rameshserver:~$ sudo apt install unattended-upgrades -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
unattended-upgrades is already the newest version (2.9.1+nmu4ubuntu1).
unattended-upgrades set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Enable automatic updates:



Check configuration:

```
ramesh@rameshserver:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
cat: /etc/apt/apt.conf.d/20auto-upgrades: No such file or directory
ramesh@rameshserver:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
cat: /etc/apt/apt.conf.d/20auto-upgrades: No such file or directory
```

Automatic security upgrades are provided through a package called unattended-upgrades, and provide a risk-reducing mechanism for software, because users will not have to monitor for and apply critical security upgrades manually.

3. Configure fail2ban for enhanced intrusion detection?

Ans: Fail2Ban Configuration

Installation:

```

Unpacking python3-pyinotify (0.9.6-2ubuntu1) ...
Selecting previously unselected package whois.
Preparing to unpack .../whois_5.5.22_amd64.deb ...
Unpacking whois (5.5.22) ...
Setting up whois (5.5.22) ...
Setting up python3-pyasyncore (1.0.2-2) ...
Setting up fail2ban (1.0.2-3ubuntu0.1) ...
/usr/lib/python3/dist-packages/fail2ban/tests/fail2banregex testcase.py:224: SyntaxWarning: invalid escape sequence '\s'
  "1490349000 test failed.dns.ch", ""\s*test <F-ID>\s+</F-ID>"
/usr/lib/python3/dist-packages/fail2ban/tests/fail2banregex testcase.py:435: SyntaxWarning: invalid escape sequence '\S'
  '^'+prefix+'<F-ID>User <F-USER>\S+</F-USER></F-ID> not allowed\n'
/usr/lib/python3/dist-packages/fail2ban/tests/fail2banregex testcase.py:443: SyntaxWarning: invalid escape sequence '\S'
  '^'+prefix+'User <F-USER>\S+</F-USER> not allowed\n'
/usr/lib/python3/dist-packages/fail2ban/tests/fail2banregex testcase.py:444: SyntaxWarning: invalid escape sequence '\d'
  '^'+prefix+'Received disconnect from <F-ID><ADDR> port \d+</F-ID>'
/usr/lib/python3/dist-packages/fail2ban/tests/fail2banregex testcase.py:451: SyntaxWarning: invalid escape sequence '\s'
  _test_variants('common', prefix="\s*\s+ sshd\[<F-MLFID>\d+</F-MLFID>\]:\s+")
/usr/lib/python3/dist-packages/fail2ban/tests/fail2banregex testcase.py:537: SyntaxWarning: invalid escape sequence '\['
  'common[prefregex="\s*\[<F-MLFID>\d+</F-MLFID>\] connect <F-CONTENT>.+</F-CONTENT>$"'
/usr/lib/python3/dist-packages/fail2ban/tests/server testcase.py:1375: SyntaxWarning: invalid escape sequence '\s'
  "{ nft -a list chain inet f2b-table f2b-chain | grep -oP '@addr-set-j-w-nft-mp\s+.*\s+Khandle\s+(\d+)\$'; } | while r
/usr/lib/python3/dist-packages/fail2ban/tests/server testcase.py:1378: SyntaxWarning: invalid escape sequence '\s'
  "{ nft -a list chain inet f2b-table f2b-chain | grep -oP '@addr-set-j-w-nft-mp\s+.*\s+Khandle\s+(\d+)\$'; } | while
/usr/lib/python3/dist-packages/fail2ban/tests/server testcase.py:1421: SyntaxWarning: invalid escape sequence '\s'
  "{ nft -a list chain inet f2b-table f2b-chain | grep -oP '@addr-set-j-w-nft-ap\s+.*\s+Khandle\s+(\d+)\$'; } | while r
/usr/lib/python3/dist-packages/fail2ban/tests/server testcase.py:1424: SyntaxWarning: invalid escape sequence '\s'
  "{ nft -a list chain inet f2b-table f2b-chain | grep -oP '@addr-set-j-w-nft-ap\s+.*\s+Khandle\s+(\d+)\$'; } | while
Created symlink /etc/systemd/system/multi-user.target.wants/fail2ban.service → /usr/lib/systemd/system/fail2ban.service.
Setting up python3-pyinotify (0.9.6-2ubuntu1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

```

Enable and Start:

```

ramesh@rameshserver:~$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable fail2ban

```

Check Status:

```

ramesh@rameshserver:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
- Jail list:  sshd

```

To protect against brute force attacks, Fail2Ban was both installed and configured to monitor login/logon authentication logs, and to temporarily block the IP address of the attacker when the attacker demonstrates evidence of malicious activity

4. Create a security baseline verification script (` security-baseline.sh `) that runs on the server (executed via SSH) and verifies all security configurations from Phases 4 and 5.

```

sbuid-distupgrade
sbuid-hold
sbuid-shell
sbuid-unhold
sbuid-update
sbuid-upgrade
scide
signal-desktop
slack
slirp4netns
steam
stress-ng
surfshark
systemd-coredump
thunderbird
toybox
trinity
tup
tuxedo-control-center
userbindmount
uwsgi-core
vdens
virtiofsd
vivaldi-bin
vpns
vscode
wike
wpcorn
1 processes have profiles defined.
1 processes are in enforce mode.
   /usr/sbin/rsyslogd (782) rsyslogd
0 processes are in complain mode.
0 processes are in prompt mode.
0 processes are in kill mode.
0 processes are unconfined but have a profile defined.
0 processes are in mixed mode.
ramesh@rameshserver:~$ echo "automatic updates..."
automatic updates...
ramesh@rameshserver:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
ramesh@rameshserver:~$ echo "checking fail2Ban status..."
checking fail2Ban status...
ramesh@rameshserver:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
  - Jail list:  sshd
ramesh@rameshserver:~$ chmod +x security-baseline.sh
ramesh@rameshserver:~$ ./security-baseline.sh

```

To ensure that all security configurations in Phase Four (4) and Five (5) are consistent, a baseline verification script has been written. The script uses SSH hardening as well as Firewall rules, AppArmor enforcement, Automatic Updates and Fail2Ban status to ensure that all security configurations are identical for each machine.

5. Create a remote monitoring script (`monitor-server.sh`) that runs on your workstation, connects via SSH, and collects performance metrics from the server.?

Ans: **Remote Monitoring Script**

monitor-server.sh (Runs on Workstation)

monitor-server.sh (Runs on Workstation)

```
ramesh@192.168.56.101's password:
04:13:10 up 9 min, 2 users, load average: 0.01, 0.04, 0.01
ramesh@rameshserver:~$ echo "collecting memory usage..."
collecting memory usage...
ramesh@rameshserver:~$ ssh $SERVER "free -h"
Enter passphrase for key '/home/ramesh/.ssh/id_ed25519':
ramesh@192.168.56.101's password:
              total        used        free      shared  buff/cache   available
Mem:            3.8Gi        468Mi        3.3Gi        1.1Mi        296Mi        3.4Gi
Swap:            3.8Gi           0B        3.8Gi
ramesh@rameshserver:~$ echo "collecting disk usage..."
collecting disk usage...
ramesh@rameshserver:~$ ssh $SERVER "df -h"
Enter passphrase for key '/home/ramesh/.ssh/id_ed25519':
ramesh@192.168.56.101's password:
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            392M  1.1M  391M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv 24G   7.5G   15G  34% /
tmpfs            2.0G   0  2.0G   0% /dev/shm
tmpfs            5.0M   0  5.0M   0% /run/lock
/dev/sda2        2.0G  197M  1.6G  11% /boot
tmpfs            392M  12K  392M   1% /run/user/1000
ramesh@rameshserver:~$ echo "collecting network information ..."
collecting network information ...
ramesh@rameshserver:~$ ssh $SERVER "ip addr show"
Enter passphrase for key '/home/ramesh/.ssh/id_ed25519':
ramesh@192.168.56.101's password:
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c4:09:ac brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 85583sec preferred_lft 85583sec
    inet6 fd17:625c:f037:2:a00:27ff:fec4:9ac/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86262sec preferred_lft 14262sec
    inet6 fe80::a00:27ff:fec4:9ac/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a3:03:7a brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 metric 100 brd 192.168.56.255 scope global dynamic enp0s8
        valid_lft 385sec preferred_lft 385sec
    inet6 fe80::a00:27ff:fea3:37a/64 scope link
        valid_lft forever preferred_lft forever
```

Safe connection to the Ubuntu Linux server was successful in via SSH keys as login. The SSH host wanted a code to the user's Ed25519 secure key not the user account password, if the SSH link was made using the command: ssh. ramesh@192.168.56.101`. This shows SSH key login is accurately used, and user passwords is disabled, making hacking and fraud attacks harder.